

# Table des matières du logiciel - Kna+

|                                                                                                                                                                                                                              |   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| Cette présentation générale présente le logiciel de gestion des agents de l'État, ses fonctionnalités principales, son architecture, son installation, et les flux de travail clés pour la démonstration et la présentation. | 2 |
| 2. Architecture technique                                                                                                                                                                                                    | 2 |
| 3. Installation et exécution                                                                                                                                                                                                 | 2 |
| 4. Structure des pages principales                                                                                                                                                                                           | 2 |
| 5. Workflow des agents                                                                                                                                                                                                       | 2 |
| 6. Notifications                                                                                                                                                                                                             | 3 |
| 7. Stockage des documents                                                                                                                                                                                                    | 3 |
| 8. Détails fonctionnels                                                                                                                                                                                                      | 3 |
| 9. Points techniques spécifiques                                                                                                                                                                                             | 3 |
| 10. Présentation à montrer                                                                                                                                                                                                   | 3 |
| 11. Fichiers et composants clés                                                                                                                                                                                              | 3 |
| 12. Flux de données techniques                                                                                                                                                                                               | 4 |
| 13. Recommandations et évolutions                                                                                                                                                                                            | 4 |

# 1. Présentation générale

Le projet est une application web de gestion de dossiers agent, construite avec Next.js 15, React 19 et Prisma. Il permet de créer des agents, gérer des documents, suivre le statut des dossiers, et afficher des notifications de vérification.

## 2. Architecture technique

L'architecture repose sur Next.js App Router avec des pages React côté client et des routes API côté serveur. La logique métier est séparée entre les pages de l'interface, les APIs de gestion des données, et les helpers de stockage de fichiers. Prisma gère l'accès à la base de données et le schéma, tandis que le stockage des fichiers est directement exposé via le dossier public/uploads.

- Frontend: Next.js App Router, composants React avec `use client` pour les interactions.
- Backend: routes API dans app/api pour les agents, documents, notifications et recherche.
- Base de données: Prisma avec MariaDB/MySQL, plus fallback mémoire possible via lib/dbService.ts.
- Stockage de fichiers: dossier public/uploads, accessible statiquement via /uploads.
- Documentation PDF: génération locale avec jsPDF et Node.js.

## 3. Installation et exécution

Ce projet fonctionne dans un environnement Node.js moderne sur Windows ou Linux. La configuration minimale inclut Node.js 20+ et npm. Le démarrage local se fait via npm run dev, et les variables d'environnement principales sont configurées dans .env.local.

- Installer les dépendances: npm install
- Créer le fichier .env.local et ajouter les variables nécessaires (par exemple DATABASE\_URL, GEMINI\_API\_KEY).
- Lancer le serveur: npm run dev
- Accéder à l'application: http://localhost:3000
- Recharger le projet après modification des fichiers source.

## 4. Structure des pages principales

- app/agents/page.tsx: vue principale de la liste des agents, filtres d'état, actions de validation et paiement.
- app/agents/create/page.tsx: formulaire de création d'un nouvel agent avec direction et informations personnelles.
- app/agents/[id]/page.tsx: détail d'un dossier agent, upload de documents, statut et actions contextuelles.
- app/documents/page.tsx: gestion documentaire centralisée avec recherche par agent et direction.
- app/api/documents/upload/route.ts: API de réception multipart/form-data pour enregistrer les fichiers.

## 5. Workflow des agents

Lorsqu'un agent est créé, le dossier est automatiquement placé en statut VERIFICATION. Le workflow autorise ensuite la validation ou le rejet du dossier par l'administrateur. Les actions disponibles dans l'interface changent en fonction du statut pour éviter les incohérences.

- Création: statut initial VERIFICATION.
- Validation: statut VALIDE, badge vert et visibilité de paiement possible.
- Rejet: statut REJETE, actions limitées pour empêcher la validation.
- Paiement: état PAYE / NON\_PAYE séparé du statut de dossier.

## 6. Notifications

Les notifications métiers ciblent les dossiers en vérification. Cette stratégie améliore le suivi du processus de contrôle de conformité et met en avant les dossiers qui nécessitent une action administrative.

- Filtre de notification: statut VERIFICATION uniquement.
- Interface: badge et lien direct vers le dossier agent.
- Alerte métier: priorisation des dossiers non encore traités.

## 7. Stockage des documents

Les documents sont déposés dans public/uploads selon une arborescence direction/agent. Le dossier agent est construit à partir du nom complet et de l'identifiant pour garantir lisibilité et unicité, puis le fichier est renommé avec timestamp.

- Répertoire agent: `safeAgentName-agentId` généré par lib/documentStorage.ts.
- Exemple: public/uploads/Direction\_Generale/mary-claire-mbuyi-ag-123456/
- Normalisation: suppression des caractères spéciaux et espaces convertis en tirets.
- Téléchargement: URLs accessibles via /uploads/... sur le serveur Next.js.

## 8. Détails fonctionnels

- Recherche d'agents par nom, matricule et téléphone.
- Filtres par direction, statut de dossier et paiement.
- Affichage de cartes agents et du gestionnaire documentaire.
- Ouverture et téléchargement sécurisés des fichiers uploadés.
- Badges de statut explicites: VALIDÉ, EN VÉRIFICATION, REJETÉ.

## 9. Points techniques spécifiques

- Nom du dossier agent = nom complet + identifiant unique.
- Statut d'agent par défaut = VERIFICATION à la création.
- Notifications basées sur le statut VERIFICATION plutôt que le paiement.
- Gestion des liens de fichiers uniquement si l'URL est valide.
- UI conditionnelle sur la page de détail agent selon le statut du dossier.

## 10. Présentation à montrer

- Démonstration de la création d'agent et du statut initial VERIFICATION.
- Upload d'un document et validation de l'ouverture depuis le gestionnaire documentaire.
- Affichage des badges VALIDÉ / EN VÉRIFICATION / REJETÉ dans l'interface.
- Navigation vers les notifications de vérification.
- Consultation de l'arborescence public/uploads pour vérifier le stockage.

## 11. Fichiers et composants clés

- lib/documentStorage.ts: création des dossiers direction/agent, normalisation des noms, génération d'URL de document.
- lib/prisma.ts: configuration Prisma et initialisation du client.
- lib/dbService.ts: abstraction de l'accès aux données et fallback mémoire.
- app/api/agents/route.ts: création, lecture et mise à jour des agents avec statut par défaut VERIFICATION.
- app/api/documents/upload/route.ts: réception des fichiers multipart et stockage physique.
- app/api/agents/notifications/route.ts: notifications filtrées sur les dossiers en VERIFICATION.
- app/documents/page.tsx: interface de gestion documentaire centrale, recherche et liens de téléchargement.
- app/agents/[id]/page.tsx: détail du dossier agent, affichage des documents et actions métier.

## 12. Flux de données techniques

- Création d'agent: formulaire -> API /api/agents -> base de données -> statut VERIFICATION.
- Upload document: formulaire de fichier -> API /api/documents/upload -> stockage public/uploads -> URL publique.
- Notifications: requête API /api/agents/notifications -> filtre sur statut VERIFICATION -> affichage.
- Affichage documents: lecture des fichiers stockés et génération d'URL via utils de storage.

## 13. Recommandations et évolutions

- Ajouter un stockage cloud (Azure Blob / S3) pour la production.
- Mettre en place une authentification et une autorisation plus fine.
- Ajouter des tests automatisés pour les APIs et le service de stockage de documents.
- Générer des miniatures ou des aperçus pour les documents uploadés.